



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Experiment Number 1

Name : Atharva Ambilwade

Class : A

Division : A2

Course: Programming with Java

SEM-III

A.Y. 2026-2027



Subject: Programming with Java

ASSIGNMENT No : 1

TITLE : JDK/JRE/JVM, variables, methods and constructors. and demonstrate the implementation of the concepts through suitable examples.

Problem Statement

JDK/JRE/JVM, variables, methods and constructors. and demonstrate the implementation of the concepts through suitable examples.

Learning Objectives

Understand the theoretical concepts.

Theory

Java environment setup, JDK/JRE/JVM, variables, methods and constructors. forms an important component of Java programming. Students should understand the underlying concepts, architecture, advantages, use cases, and implementation methodology. The exercise should focus on understanding the design principles, coding practices, execution flow, and practical applications of these concepts in software development. Detailed study should include syntax, APIs, execution process, and real-world applications.

The main concept behind Java's setup is platform independence. Unlike languages that compile code directly on operating system, Java compiles code in a virtual machine. Java environment setup, JDK/JRE/JVM, variables, methods and constructors. forms an important component of Java programming.

JDK (Java Development Kit):

The JDK is a software package used to develop Java applications. It provides all the tools needed to write, compile, debug, and run Java programs. It includes the Java compiler (`javac`), the Java Runtime Environment (JRE), a debugger, and other development tools.

JRE (Java Runtime Environment):

The JRE is the software required to run Java applications on a computer. It includes the Java Virtual Machine (JVM) and the necessary libraries to execute Java programs. However, it does not include tools for developing or compiling Java applications.

JVM (Java Virtual Machine):

The JVM is a virtual machine that runs Java programs. It converts Java bytecode into machine code that the computer can understand and execute. It also manages memory and enables Java programs to run on different operating systems without any changes, making Java platform-independent.

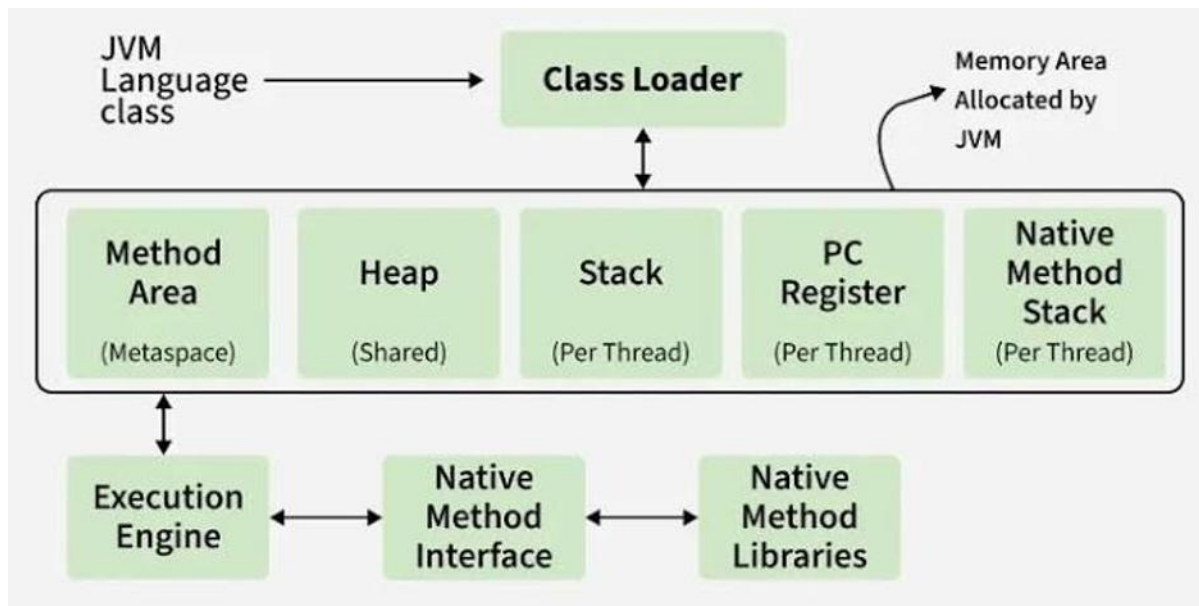


Fig 1.1 : JVM architecture

Figure 1.1 demonstrates a JVM architecture. The components are as follows :

1. Class Loader

The Class Loader loads the required Java class files (.class) into the JVM before execution.

2. Method Area (Metaspace)

Stores class information, method details, and metadata. It is shared by all threads.

3. Heap Memory (Shared)

Stores all objects and arrays created during program execution. The JVM automatically removes unused objects using Garbage Collection.

4. Stack Memory (Per Thread)

Stores method calls, local variables, and temporary data. Each thread has its own stack.

5. PC (Program Counter) Register (Per Thread)

Keeps track of the current instruction being executed by each thread.

6. Native Method Stack (Per Thread)

Stores information for methods written in languages like C or C++ that are called by Java.

7. Execution Engine

Executes Java bytecode by converting it into machine code. It uses the Interpreter and JIT Compiler to improve performance.

8. Java Native Interface (JNI)

Allows Java programs to communicate with native programs and libraries written in other languages.

9. Native Method Libraries

These are external libraries containing native code that Java programs can access through JNI.

Working of JVM

When a Java program is executed, the Class Loader loads the required class files into the JVM. The class information is stored in the Method Area, while objects and arrays are stored in the Heap Memory. Each thread is provided with its own Stack, PC Register, and Native Method Stack to manage method execution and thread-specific operations. The Execution Engine then converts the Java bytecode into machine code and executes the program. During execution, the JVM automatically manages memory by removing unused objects through Garbage Collection. If the program needs to use code written in other languages such as C or C++, it accesses the required Native Method Libraries through the Java Native Interface (JNI).

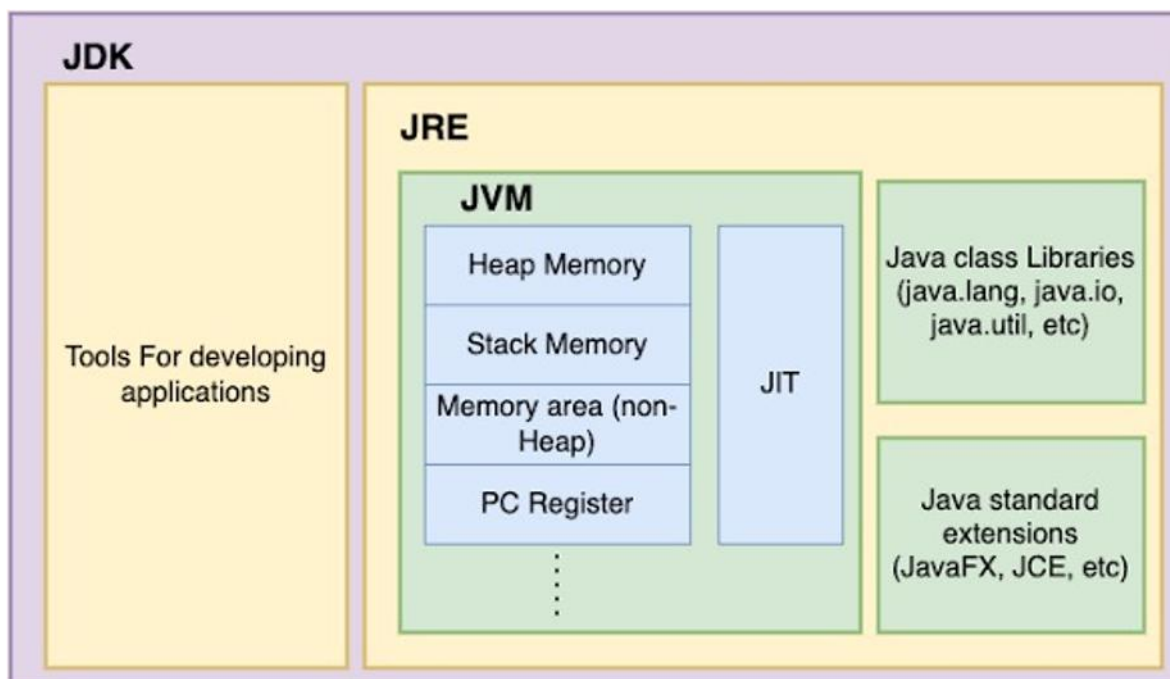


Fig 1.2 : Relationship Between JDK, JRE, and JVM

Figure 1.2 illustrates the relationship between the JDK, JRE, and JVM. The JDK is the complete development package that contains the JRE, while the JRE includes the JVM, making it responsible for running Java applications. Within the JVM, components such as Heap Memory, Stack Memory, the PC Register, and the JIT (Just-In-Time) Compiler work together to manage memory and execute Java programs efficiently. In addition, Java Class Libraries and Standard Extensions provide pre-built classes and APIs that simplify application development. This layered architecture separates development, execution, and memory management, making Java both platform-independent and easy to develop, maintain, and run across different operating systems.



Conclusion:

This assignment helped in understanding the Java programming environment, including the roles of **JDK**, **JRE**, and **JVM**, and how they work together to develop and run Java applications

Questions:

Q1) What are the advantages of using Java variables and methods?

Ans) Advantages of Using Java Variables and Methods are as follows :

- Improves code readability by using meaningful variable and method names.
- Promotes code reusability since methods can be called multiple times without rewriting code.
- Makes debugging easier by organizing logic into smaller, manageable methods.
- Reduces code duplication, resulting in cleaner and more efficient programs.
- Enhances maintainability, making it easier to update or modify the code.
- Supports modular programming by dividing a program into independent methods.
- Improves memory management by storing data efficiently using variables.
- Increases program flexibility as variable values can change during execution without altering the program logic.

Q2) How can this implementation be improved?

Q3) What are the real-world applications?

Ans) Java is widely used to build secure, reliable, and platform-independent applications across various industries. Some of its major real-world applications are:

- Web Applications – Builds dynamic and scalable websites and web services.
- Android Applications – Develops mobile apps for Android devices.
- Enterprise Applications – Powers banking, e-commerce, healthcare, and business management systems.
- Cloud Computing – Creates scalable cloud-based applications and services.
- Big Data Applications – Processes and analyzes large datasets using frameworks like Hadoop and Spark.